CGG 2026 Configurations for Visual Studio 2026 - Community Edition

In this document, we will describe the project configurations and dependencies that are required for using the CGG 2026 source code and to run your own projects.

Additionally, we will provide some notes regarding common issues.

```
<!-- When viewing this markdown source, remember that $ is used in
math markdown, and as such, should be escaped, so \$ just means $ -->
```

Downloading and installing Visual Studio 2026 - Community Edition

- Head to https://visualstudio.microsoft.com/downloads/
- Download **Visual Studio Community Edition**.
- Run the installation wizard.
- When asked to select what components to install, on the **Workloads** window, select **Desktop Development with C++**.
- Click *Install*.

Manual Installation

It is important to setup the project as a **Console Application** so that there are no unexpected errors regarding different signatures for the entry functions (the main function).

1) Creating a new Console App Project from scratch

In order to create a new Console App Project from scratch, you can simply use the wizard that appears when you open up *Visual Studio*, or you can do it from *Visual Studio*'s menu. This will include code for a "Hello World!" program.

- Select *File -> New -> Project* from the menu bar
- Select *Console App*
- Insert the *Project Name*

- Select the *Project Location*
- Insert the *Solution Name*
- Click *Create*

2) **Downloading the necessary libraries**

First, you should create a **libraries** folder on the same folder of the Solution you have created.

Then, create a folder for each library on the **libraries** folder:

- *libraries/glm*
- *libraries/glfw*
- *libraries/glew*
- *libraries/assimp*

Then you can download the following libraries.

- **GLM** https://sourceforge.net/projects/glm.mirror/
    - Scroll down to *Project Activity*
    - Click on *Released/* **version** */glm-***version**.zip.
    - where **version** is the current version. Example: 1.0.3
    - Extract the contents of the zip you have just downloaded to *libraries/glm*
- **GLFW** https://www.glfw.org/download
    - Click on the binaries that correspond to your machine:
        - **64-bit Windows binaries**
        - **32-bit Windows binaries**
    - Extract the contents of the zip you have just downloaded to *libraries/glfw*
- **GLEW** https://glew.sourceforge.net/
    - Head to *Downloads*
    - Click on *Binaries* **Windows 32-bit and 64-bit**
    - Extract the contents of the zip you have just downloaded to *libraries/glew*
- **Assimp** https://kimkulling.itch.io/the-asset-importer-lib
    - Click *Download Now*
    - This library is free. **You do not need to donate.**
    - Click *No thanks, just take me to the downloads*
    - Click *Download*
    - Run the installer.
    - Locate the *Assimp* folder on your machine and move/copy its contents to *libraries/assimp*

- **Remember to download the pre-compiled binaries for your machine**.
- **Do not download source code**.

OpenGL should already be installed on your system, and, as such, you do not need to install it or copy it into a project.

3) Library Setup

During this section you will configure the project's properties.

Open your project's properties:

- Right-click the project's name
- Select *Properties* (should be the last item in the right-click menu)

The properties are applied to a specific *Configuration*, which can be *Debug* or *Release*. You should setup both configurations at once by:

- Click on *Configuration* at the top of the window.
- Select **All Configurations**

Now you can edit the following project properties.

Additional Include Directories

- Select Configuration Properties -> C/C++ -> General -> Additional Include Directories
- Add $(SolutionDir)libraries\glew\include;
- Add $(SolutionDir)libraries\glfw\include;
- Add $(SolutionDir)libraries\glm;
- Add $(SolutionDir)libraries\assimp\include;

Additional Library Directories

- Select *Configuration Properties -> Linker - General -> Additional Library Directories*
- Add $(SolutionDir)libraries\glfw\lib-vc2026;
- Add $(SolutionDir)libraries\glew\lib\Release\x64;
- Add $(SolutionDir)libraries\assimp\lib\x64;

Additional Dependencies

- Select *Configuration Properties -> Linker -> Input -> Additional Dependencies*
- Add opengl32.lib;
- Add glew32.lib;
- Add glfw3dll.lib;
- Add assimp-vc145-mt.lib;

Additional Information

Copying the required libraries to the output directories

The current workflow requires manually copying some libraries (dll) into the output directory.

These libraries are located at:

- glew32 -> libraries\glew\bin\Release\x64\glew32.dll
- glfw3 -> libraries\glfw\lib-vc2026\glfw3.dll
- assimp -> libraries\assimp\bin\x64\assimp-vc145-mt.dll

An alternative to manually copying the libraries is setting up a *Post-Build Event*, which can be done in the project's properties:

- Right-click the project's name
- Select *Properties* (should be the last item in the right-click menu)
- Navigate to *Configuration Properties -> Build Events -> Post-Build Event -> Command Line*
- Add *xcopy /y $(SolutionDir)libraries\glew\bin\Release\x64\glew32.dll $(OutDir)*
- Add *xcopy /y $(SolutionDir)libraries\glfw\lib-vc2026\glfw3.dll $(OutDir)*
- Add *xcopy /y $(SolutionDir)libraries\assimp\bin\x64\assimp-vc145-mt.dll $(OutDir)*

```
<!--
For folders with spaces in their names try putting quotes:
xcopy /y "$(SolutionDir)libraries\glew\bin\Release\x64\glew32.dll"
"$(OutDir)"
-->
```

Library Paths

We are using the path **$(SolutionDir)libraries** to signify that each of the used libraries is placed under a folder named **libraries** of the solution's root directory. The macro **$(SolutionDir)** is just a *Visual Studio* macro that points to the solution's directory, no matter where it is located - basically just solves the issues that can arise from having fixed paths and migrating projects from one machine to another.

Error : Unsupported C++ feature

In case you run into an error where a library uses an unsupported C++ feature (for example, some feature from C++17), you can change the language standard in the project properties.

- Right-click the project's name
- Select *Properties* (should be the last item in the right-click menu)
- Navigate to *Configuration Properties -> C/C++ -> Language -> C++ Language Standard*
- Select **ISO C++17 Standard (/std:c++17)**

Linux source code to Windows Issues

Error C2079 'oss' uses undefined class 'std::basic_ostringstream<char,std::char_traits<char>,std::allocator<char>>'

- Simply add *#include <sstream>* to the includes of the code

Applications with multiple executables (entry points)

For solutions with multiple entry points, remember to create different projects in the solution (one for each entry point), and to make sure all the dependencies and linking are sorted for the multiple projects in the solution.

One solution to this, if starting from existing code, is to simply start by creating multiple projects in subfolders of the solution's directory (with the necessary source code files). Then create a new project from existing code in the root, add the previously created projects to this new solution, then delete the dummy project that was created by default in this new solution. Then you can delete the other solutions that were created for the subfolders (they are no longer needed).

When creating a project from scratch, simply add new projects to the solution as subdirectories.